

Computer Networks

CA1 Report

Shayan Maleki 810102515

Mani Hosseini 810102552

Architecture

1) Network Architecture & Pattern

We are using a Client-Server system that the server acts as a central node and we have several client nodes that connect to the server via TCP protocol and give commands to the server .

We use TCP because we need reliable and point to point connection between each client and server .

2) Concurrency and IO Handling

The system that we have implemented is an event driven and single threaded architecture to handle multiple simultaneous client connections. Instead of using multiple threads or forking new processes we use IO multiplexing by using select function . what it essentially does is that it keeps switching between the sockets to monitor their data to see if there is a new data on a socket .

It continuously monitors them until one of them sends a new data.

This approach enables the server to concurrently manage new connection requests, real-time chat messages, and file transfers with high efficiency and minimal system overhead, avoiding the complexities of thread synchronization.

3) Software Architecture:

We have two core classes (client, server) that handle everything .

They have two methods setup() and run() :

Setup(): it essentially creates the sockets and set their options and calls the listen() function (for server) so that the sockets would be ready to get data to them and send the responses.

```
void Server::setup() {  
    server_fd = socket(AF_INET, SOCK_STREAM, 0);  
  
    int opt = 1;  
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));  
  
    sockaddr_in tcp_addr{};  
    tcp_addr.sin_family = AF_INET;  
    tcp_addr.sin_addr.s_addr = INADDR_ANY;  
    tcp_addr.sin_port = htons(SERVER_TCP_PORT);  
  
    bind(server_fd, (struct sockaddr*)&tcp_addr, sizeof(tcp_addr));  
    listen(server_fd, 10);  
}
```

Run(): it implements the select function and calls the proper private functions based on the socket transponding the data.

```

void Server::run() {
    fd_set read_fds;
    int max_fd;

    while (true) {
        FD_ZERO(&read_fds);
        FD_SET(server_fd, &read_fds);
        max_fd = server_fd;

        for (size_t i = 0; i < client_fds.size(); i++) {
            FD_SET(client_fds[i], &read_fds);
            if (client_fds[i] > max_fd) {
                max_fd = client_fds[i];
            }
        }

        select(max_fd + 1, &read_fds, nullptr, nullptr, nullptr);

        if (FD_ISSET(server_fd, &read_fds)) {
            accept_client();
        }

        for (size_t i = 0; i < client_fds.size(); i++) {
            int fd = client_fds[i];
            if (FD_ISSET(fd, &read_fds)) {
                if (!handle_receive(fd)) {
                    client_fds.erase(client_fds.begin() + i);
                    i--;
                }
            }
        }
    }
}

```

How to run

At first you need to use the makefile and typing "make all" to make the server.out and client.out files then here is the syntax for each command:

MSG <message>

PM <recievers username> <message>

USERS

LIST

GET <filename>

PUT <filename>

QUIT

By sending any of these commands in these syntaxes you will be able to interact with this application and also screenshots are also included.

Commands Implementation

First of all, before explaining the command implementation flow, we need to know more about the software architecture design. As explained before, we have a `run()` function in our core classes that starts the application. In the server's `run` function, after using `select`, we have to make a distinction between the server's own socket (`server_fd`) and the client sockets. This is because the server socket's job is to wait for a new client to connect and call the `accept` function, but activity on a client socket means that there is data coming from one of the connected clients.

```
select(max_fd + 1, &read_fds, nullptr, nullptr, nullptr);

if (FD_ISSET(server_fd, &read_fds)) {
    accept_client();
}

for (size_t i = 0; i < client_fds.size(); i++) {
    int fd = client_fds[i];
    if (FD_ISSET(fd, &read_fds)) {
        if (!handle_receive(fd)) {
            client_fds.erase(client_fds.begin() + i);
            i--;
        }
    }
}
```

When `select` says the `server_fd` is ready, we call the `accept_client()` function. This function simply accepts the new connection and adds the new socket number to our `client_fds` vector so we can monitor it in the next loops.

```
void Server::accept_client() {
    sockaddr_in client_addr{};
    socklen_t client_len = sizeof(client_addr);
    int new_client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &client_len);
    client_fds.push_back(new_client_fd);
}
```

If `select` finds activity on a client socket, we call the `handle_receive()` function.

First, it reads the data using `recv`. If the client disconnected (`valread <= 0`), it clears their data and closes the socket.

If there is data, it checks the `active_uploads` map. If the client is currently in the middle of uploading a file, the server writes the bytes directly to the file stream. If the client is not uploading a file, it means the data is a normal text command. So, it sends the text to the `command_handler` to parse it into a `Command` object, and then passes it to the `execute_command()` function.

```

if (active_uploads.count(client_fd)) {
    UploadState& state = active_uploads[client_fd];

    state.file_stream->write(buffer, valread);
    state.received_size += valread;

    if (state.received_size >= state.total_size) {
        state.file_stream->close();
        delete state.file_stream;

        std::string uploader_name = online_users_by_fd[client_fd]->get_username();
        File* new_file = new File(state.filename, state.total_size, uploader_name);
        files_by_name[state.filename] = new_file;

        active_uploads.erase(client_fd);

        std::string success_msg = "File upload complete.\n";
        send(client_fd, success_msg.c_str(), success_msg.length(), 0);
    }
    return true;
} else {
    std::string message(buffer, valread);
    Command cmd = command_handler.parse(message);
    return execute_command(client_fd, cmd);
}
}

```

The `execute_command()` function works as a router. It takes the parsed `Command` object and uses a simple switch statement based on the command type. It calls the right function for that specific command (like `handle_register` or `handle_PUT`). If the command type is unknown, it sends an “invalid command” error back to the client.

Finally, to manage the state of the server and the data of all users, we used several private maps in the `Server` class:

users_by_username: A map that links a username string to a `User` object. It stores all the users that have registered on the server.

online_users_by_fd: A map that links a client’s socket (fd) to their `User` object. It only stores the users who are successfully logged in right now.

files_by_name: A map that stores information about all the files uploaded to the server. It uses the `File` class to keep the filename, size, and uploader name.

active_uploads: A map that links a client’s socket to an `UploadState` struct. It helps the server remember if a client is currently uploading a file, where to save it, and how many bytes are received so far.

Now to proceed to the commands :

MSG :

after a client sends a MSG command the commandhandler has successfully parsed the message and stored the message contents into a vector.

The handle_MSG() function gathers this content and make a string containing the message and its sender and then passes it into the broadcast function that iterates through the online users by fd map and sends the message to each one of them EXCEPT for the current fd that is sending the message.

```
void Server::handle_MSG(int client_fd, const std::vector<std::string>& args) {
    if (args.empty()) {
        std::string error_msg = "Please enter a message\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    if (online_users_by_fd.count(client_fd) == 0) {
        std::string error_msg = "You are not logged in\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    std::string full_message;
    for (int i = 0; i < args.size(); ++i) {
        full_message += args[i];
        if (i < args.size() - 1) {
            full_message += " ";
        }
    }

    std::string sender_username = online_users_by_fd[client_fd]->get_username();
    std::string formatted_msg = sender_username + ": " + full_message + "\n";

    broadcast_message(client_fd, formatted_msg);
}
```

NOTE : the broadcast function sends the message via tcp protocol to each client one by one it does NOT use udp protocol

PM:

The process is pretty similar to the MSG command the difference is that we have to do more error handling and make sure the receiver is also online

The rest is similar and after retrieving the receiver's socket fd we use send to send the formatted message to them:

```
void Server::handle_PM(int client_fd, const std::vector<std::string>& args) {
    if (args.size() < 2) {
        std::string error_msg = "Wrong syntax : PM <username> <message>\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    if (online_users_by_fd.count(client_fd) == 0) {
        std::string error_msg = "You are not logged in\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    std::string receiver_username = args[0];

    if (users_by_username.count(receiver_username) == 0) {
        std::string error_msg = "User does not exist\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    User* receiver_user = users_by_username[receiver_username];

    if (!receiver_user->get_is_online()) {
        std::string error_msg = "User is not online\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    std::string full_message;
    for (size_t i = 1; i < args.size(); ++i) {
        full_message += args[i];
        if (i < args.size() - 1) {
            full_message += " ";
        }
    }

    std::string sender_username = online_users_by_fd[client_fd]->get_username();
    std::string formatted_msg = sender_username + ": " + full_message + "\n";

    int receiver_fd = receiver_user->get_fd();
    send(receiver_fd, formatted_msg.c_str(), formatted_msg.length(), 0);
}
```

USERS & LIST :

The implementation of these commands are pretty straight forward we just had to loop through the online users and files map to print the users and files :

```
void Server::handle_USERS(int client_fd) {
    if (online_users_by_fd.count(client_fd) == 0) {
        std::string error_msg = "You are not logged in\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    std::string user_list = "Online users:\n";
    for (auto it = online_users_by_fd.begin(); it != online_users_by_fd.end(); ++it) {
        user_list += "- " + it->second->get_username() + "\n";
    }

    send(client_fd, user_list.c_str(), user_list.length(), 0);
}

void Server::handle_LIST(int client_fd) {
    if (online_users_by_fd.count(client_fd) == 0) {
        std::string error_msg = "You are not logged in\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    if (files_by_name.empty()) {
        std::string empty_msg = "No files available\n";
        send(client_fd, empty_msg.c_str(), empty_msg.length(), 0);
        return;
    }

    std::string file_list = "Uploaded files:\n";
    for (auto it = files_by_name.begin(); it != files_by_name.end(); ++it) {
        file_list += "- " + it->second->get_name() + " (" +
            std::to_string(it->second->get_size()) + " bytes) - Uploaded by: " +
            it->second->get_uploader() + "\n";
    }

    send(client_fd, file_list.c_str(), file_list.length(), 0);
}
```

PUT:

The main goal of this project was to implement these file transfer commands. TCP is a stream protocol. This means it does not understand where a message starts and where it ends. We just receive a continuous flow of bytes. Because of this we must know the exact size of the file before we start receiving it.

To solve this TCP problem we created a handshaking process. First the client sends a text command with the file name and the file size. The server reads this and prepares itself. Then the server sends a ready message back to the client. After that the client starts sending the actual file bytes.

We cannot load a very large file entirely into the computer RAM all at once. It would crash the program. So we are forced to break the file into smaller chunks. The client sends the file piece by piece. The server receives these small pieces and writes them directly to the hard drive.

Our server uses the select function to handle multiple clients at the same time. This means the server jumps between different sockets to read data. Because of this we must have a state to remember what each socket is doing. We need to know if a specific socket is currently transferring a file. If it is uploading a file the server must understand that the incoming data is a file chunk and not a normal text command. We created the UploadState struct for this exact reason.

```
struct UploadState {  
    std::string filename;  
    long total_size;  
    long received_size;  
    std::ofstream* file_stream;  
};
```

When the server receives the PUT command it calls the handle_PUT function. Here the server creates a new UploadState for the client. It opens a new file stream in the uploaded files folder. It saves the total size and sets the received size to zero. Finally it sends the ready message to the client.

```

void Server::handle_PUT(int client_fd, const std::vector<std::string>& args) {

    if (online_users_by_fd.count(client_fd) == 0) {
        std::string error_msg = "You are not logged in\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    std::string filename = args[0];
    long total_size = std::stol(args[1]);

    UploadState state;
    mkdir("uploaded files", 0777);
    state.filename = filename;
    state.total_size = total_size;
    state.received_size = 0;
    state.file_stream = new std::ofstream("uploaded files/"+filename, std::ios::binary);

    active_uploads[client_fd] = state;

    std::string ready_msg = "READY_TO_SEND_FILE\n";
    send(client_fd, ready_msg.c_str(), ready_msg.length(), 0);
}

```

After this the client starts sending the raw bytes of the file. In the `handle_receive` function the server checks the active uploads map. If the client is in this map the server knows the data is for a file. It writes the bytes directly to the file stream. It adds the received bytes to the received size variable. When the received size reaches the total size the server closes the file stream and removes the state. It then saves the file information in the files by name map.

```

if (active_uploads.count(client_fd)) {
    UploadState& state = active_uploads[client_fd];

    UploadState &state write(buffer, valread);
    state.received_size += valread;

    if (state.received_size >= state.total_size) {
        state.file_stream->close();
        delete state.file_stream;

        std::string uploader_name = online_users_by_fd[client_fd]->get_username();
        File* new_file = new File(state.filename, state.total_size, uploader_name);
        files_by_name[state.filename] = new_file;

        active_uploads.erase(client_fd);

        std::string success_msg = "File upload complete.\n";
        send(client_fd, success_msg.c_str(), success_msg.length(), 0);
    }
    return true;
}

```

GET:

The GET command works very similar to the PUT command but in reverse. We still need handshaking because the client needs to know the file size before receiving the bytes. When the server receives the GET command it runs the handle_GET function. It searches for the file in the files by name map. If the file exists the server sends the file size and name to the client.

```
void Server::handle_GET(int client_fd, const std::vector<std::string>& args) {
    if (args.empty()) {
        std::string msg = "wrong syntax :GET <filename>\n";
        send(client_fd, msg.c_str(), msg.length(), 0);
        return;
    }

    if (online_users_by_fd.count(client_fd) == 0) {
        std::string error_msg = "You are not logged in\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    std::string filename = args[0];

    if (files_by_name.count(filename) == 0) {
        std::string error_msg = "File does not exist\n";
        send(client_fd, error_msg.c_str(), error_msg.length(), 0);
        return;
    }

    File* requested_file = files_by_name[filename];
    long file_size = requested_file->get_size();

    std::string info_msg = "File_inof " + filename + " " + std::to_string(file_size) + "\n";
    send(client_fd, info_msg.c_str(), info_msg.length(), 0);
}
```

After receiving this info the client creates an empty file and sends a ready message to the server. The server reads this ready message and starts sending the file. It opens the file from the uploaded files folder. It reads the file piece by piece and sends each piece to the client. We do this chunk by chunk so we do not use too much RAM.

```
void Server::handle_READY_FOR_GET(int client_fd, const std::vector<std::string>& args) {
    if (args.empty()) {
        return;
    }

    std::string filename = args[0];
    std::ifstream file(filename, std::ios::binary);

    char buffer[1024];
    while (file.read(buffer, sizeof(buffer)) || file.gcount() > 0) {
        if (send(client_fd, buffer, file.gcount(), 0) < 0) {
            break;
        }
    }

    file.close();
}
```

Usage screenshots

At first we have made 4 users :shayan , mani , ali , hassan using register command :

```
shayan@DESKTOP-60C68B7: /mnt.  
REGISTER shayan 1234  
Registered successfully
```

Then we Login by shayan and mani to see check the USERS command:

```
LOGIN shayan 1234  
Login successful  
USERS  
Online users:  
- shayan  
- mani
```

Then we will try the PM command on mani which is a online user and hassab which is a offline user

We also log in to ali to check that the PM only sends to mani:

```
PM hassan this is a test message  
User is not online
```

```
PM mani this is a message to mani
```

```
Login successful  
shayan: this is a message to mani
```

```
Registered successfully  
LOGIN ali 1234  
Login successful  
|
```

As we can see the message didn't get sent to hassan and also when we sent a message to mani it only got sent to mani .

Now we log in to hassan as well and try the MSG command:

```
Login successful  
shayan: this is a message to mani  
shayan: this is a message to everybody  
|
```

```
LOGIN ali 1234  
Login successful  
shayan: this is a message to everybody  
|
```

```
LOGIN hassan 1234  
Login successful  
shayan: this is a message to everybody  
|
```

As we can see everybody got it.

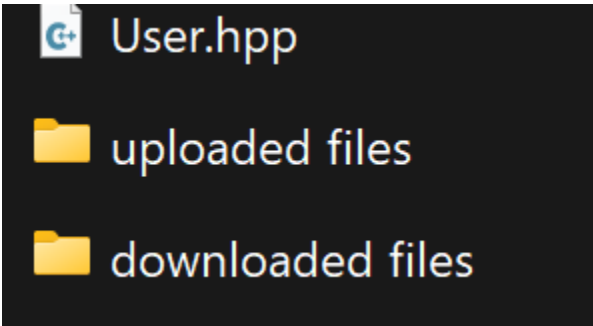
Now we proceed to GET and PUT

the PUT makes a “uploaded files” dir on the servers location and uploads the file there and the GET make a “downloaded files” dir on the clients location and downloads the files there here are the screenshots:

```
PUT ca1.pdf
READY_TO_SEND_FILE
File sent completely.
File upload complete.
```

```
LOGIN mani 1234
Login successful
GET ca1.pdf

Download completed
```



User.hpp

uploaded files

downloaded files

As we can see the directories are created successfully .